DATA100 Project

Adil Sami, 169095647

2026-04-09

1 Introduction

1.1 Context and Data Source

Steam, a gaming platform developed by Valve corporation, is the world's leading platform for PC gaming. It distributes over 100,000 games featuring a vast selection of genres, languages, and price ranges. As such, the abundant data pertaining to these games can provide insights into trends in gaming success. This data set was chosen due to its large size as well as its relevance, as the gaming industry continues to grow with each passing year. The data set contains 122,611 observations of 40 variables. This data was sourced from kaggle.com. The data was collected through the Steam Web API as well as collected from steamspy.com, a 3-rd party service that also collects data through Steam's API and makes accurate estimations of metadata.

1.2 Data Overview

Although there are 40 variables, about 22 variables can possibly provide meaningful insights. Among them is the target variable of this analysis, `Peak CCU`, which is the maximum recorded concurrent player count on a game. Due to the large amount of variables, this analysis will focus on the following:

- Continuous variables: `Peak CCU`, `Positive`, and `Price`.
- Categorical variables: `Estimated owners`, `Developers` and `Genres`.

```r
library(tidyverse)
library(ggrastr)
library(tidymodels)
tidymodels_prefer()
```

```r
games_raw <- read_csv("games.csv")
```

2 Research Objectives

This analysis aims to uncover what makes a game viral. To do this, `Peak CCU` was chosen as the target variable. Oftentimes this metric is used as an indicator of the success of a game, which it can be, but it is more accurately a measure of the virality of a game since a game with a high `Peak CCU` does not necessarily maintain that number over its lifetime. There is interesting discussion to be had on the effects on `Peak CCU` by metrics like `Positive`, `Estimated owners`, `Price`, `Developers`, and `Genres`.

3 Data Cleaning

First and foremost, I selected columns of the data frame that I would use to try to find patterns in the data.

```
games_shorter <- games_raw |> select(Name, `Release date`, `Estimated owners`,
`Peak CCU`, Price, Discount, `DLC count`, `Supported languages`, Windows, Mac,
Linux, `Metacritic score`, `User score`, Positive, Negative, Achievements, Recommendations,
`Average playtime forever`, `Median playtime forever`, Developers, Publishers, Genres)
```

Next, I removed all games without any reviews. This was done to filter out playtests, games with missing values, and extremely unpopular games that would otherwise skew results. Other methods of filtering were attempted but due to missing values, valuable information was being lost. With this, there are 64,806 remaining observations.

```
games_smaller <- games_shorter |> filter(Positive>0 & Negative>0)
```

Additionally, one particular missing value was causing trouble, so I consulted steamdb.info (another service similar to steamspy) for a more accurate owner range for the game "Marvel Rivals". This was important to fix because "Marvel Rivals" has one of the highest `Peak CCU`. Also, I removed "Wallpaper Engine" because it had a high `Peak CCU` but doesn't count as a game. The only reason for its high `Peak CCU` is likely because the application runs when people use their wallpapers, not because many people are actively using the application.

```
games_smaller$`Estimated owners`[games_smaller$Name=="Marvel Rivals"] <- "20000000 - 50000000"
games_smaller <- games_smaller|>filter(Name!="Wallpaper Engine")
```

To continue cleaning things up, I created a function to rewrite the range of owners in a more readable format.

```
clean_range<-function(data){
  case_when(
    data=="0 - 20000" ~ (data<-"0 - 20k"),
    data=="20000 - 50000" ~ (data<-"20k - 50k"),
    data=="50000 - 100000" ~ (data<-"50k - 100k"),
    data=="100000 - 200000" ~ (data<-"100k - 200k"),
    data=="200000 - 500000" ~ (data<-"200k - 500k"),
    data=="500000 - 1000000" ~ (data<-"500k - 1M"),
    data=="1000000 - 2000000" ~ (data<-"1M - 2M"),
    data=="2000000 - 5000000" ~ (data<-"2M - 5M"),
    data=="5000000 - 10000000" ~ (data<-"5M - 10M"),
    data=="10000000 - 20000000" ~ (data<-"10M - 20M"),
    data=="20000000 - 50000000" ~ (data<-"20M - 50M"),
    data=="50000000 - 100000000" ~ (data<-"50M - 100M"),
    data=="100000000 - 200000000" ~ (data<-"100M - 200M")
  )
}
```

Finally, I modified columns and created new ones to correct data types and overall make it easier to access data. Some columns were cleaned up, but didn't end up providing useful information for this analysis (`Supported languages`, `User score`, `Release Year`, and `Number of platforms`).

```r
games_clean_cols <- games_smaller|> mutate(
  # The target variable has a very wide range, so we use a logarithmic scale
  log_ccu = log(`Peak CCU`+1), # Add 1 so that 0's don't result in log(0)=-Inf

  # Some `User score` had missing values, but this can easily be calculated as the
  # percentage of Positive reviews.
  `User score` = round((Positive/(Positive+Negative))*100),

  # Extract max estimated owners as an integer. This will be useful in ordering factors.
  `Estimated owners high` = as.integer(str_extract(`Estimated owners`, "\\d+$")),
  .after=`Estimated owners`,

  # Use the previously defined function to make labels easier to read.
  `Estimated owners` = clean_range(`Estimated owners`),

  # Instead of a list of supported languages, the number of them is more useful.
  `Supported languages` = lengths(str_split(`Supported languages`,",")),

  # Extract year from `Release date`
  `Release year` = year(mdy(`Release date`)),

  # Create column on if the game is free or paid
  Free = if_else(Price==0, "Free", "Paid"),

) |> mutate(

  # Combine the logical columns `Windows`, `Mac` and `Linux` into a single column giving
  # the number of platforms supported.
  `Number of platforms` = (Windows+ Mac+ Linux),
  .after=`Supported languages`

) |> select(!c(Windows, Mac, Linux, `Release date`)) # Remove unnecessary columns
```

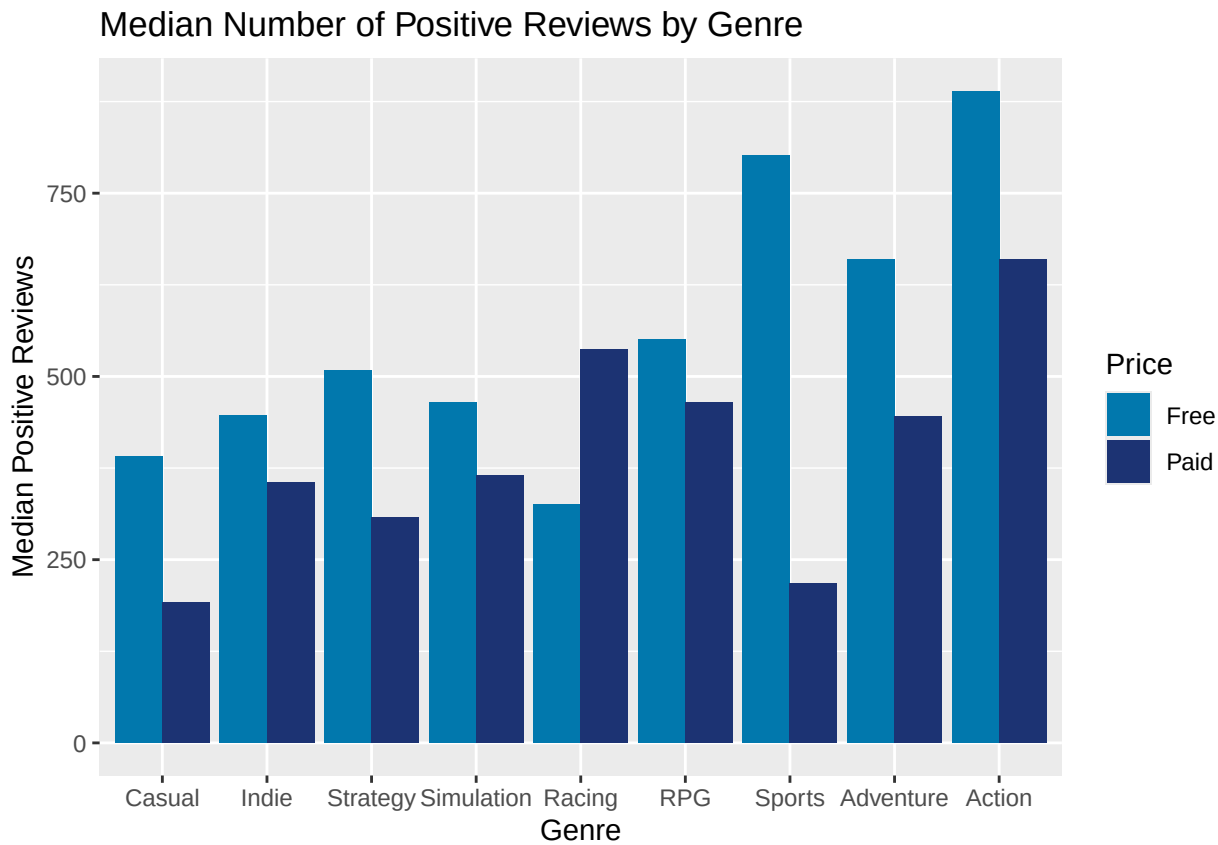
4 Exploratory Plots

4.1 Trends in Positive Reviews by Genre

The first of the exploratory plots is a bar graph of the median number of positive reviews for each genre, split by price. To plot this, I first separated the `Genres` column so that games with multiple genres would have one row for each genre. I kept only the 9 most relevant genres.

```r
games_genres<- games_clean_cols |>
  separate_longer_delim(cols=Genres, delim=",")|>
  filter(Genres %in% c("Indie", "Action", "Adventure", "Casual",
                       "Simulation","Strategy","RPG","Sports","Racing"))

avg_pos_genre <- games_genres |>
  filter(`Peak CCU`>0)|>
  group_by(Genres, Free)|>
  summarise(avg=median(`Positive`))|>
  ggplot(aes(x=fct_reorder(Genres,avg), y=(avg), fill=Free))+
```

```
  geom_col(position="dodge") +
  scale_fill_manual(values=c("#0178ad","#1c3373")) +
  genre_labs

avg_pos_genre
```



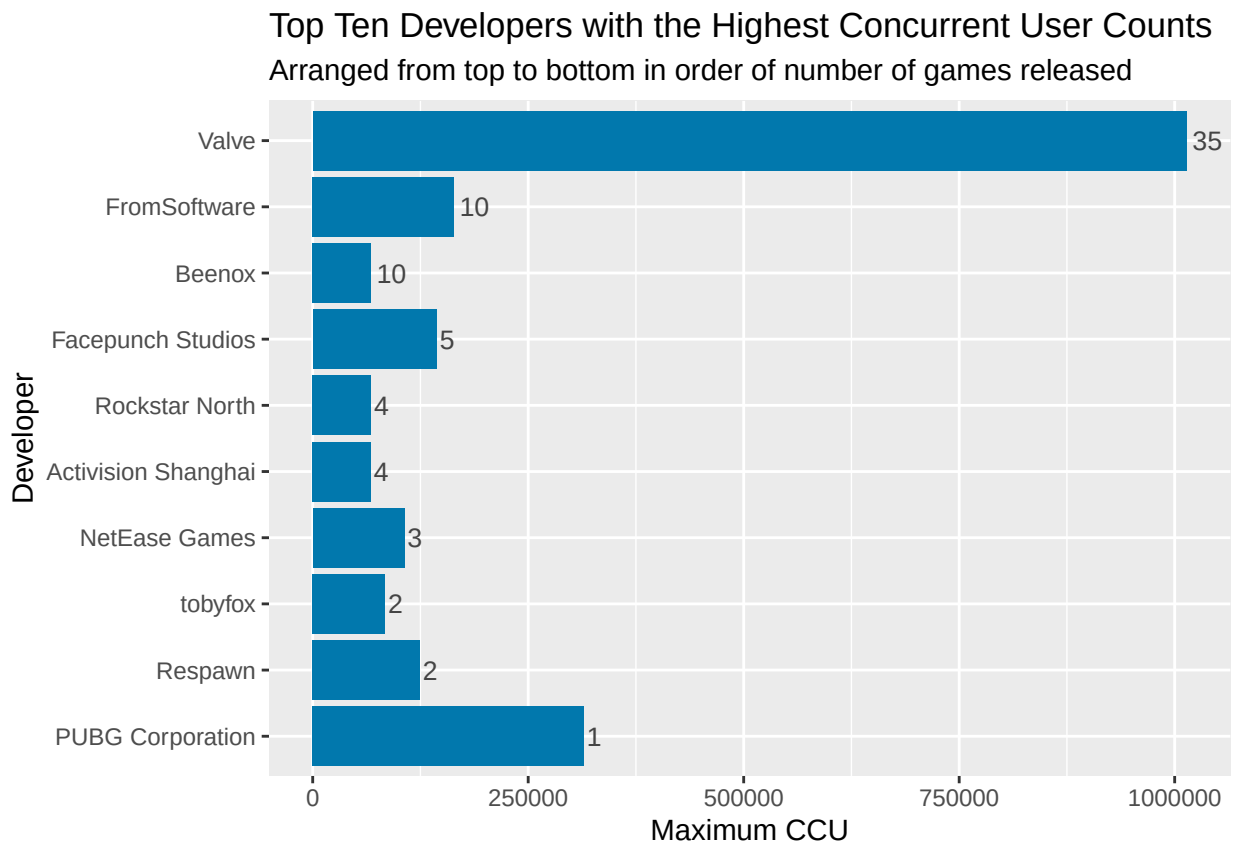
This plot reveals that free-to-play games generally gain more attention than paid games. This is because free games are more easily accessible so they get more players and thereby more positive reviews. Also, action games seem to be the most popular and in particular free ones see success the most often. The purpose of this plot was to determine if genre had a direct effect on positive reviews but it seems other factors like price have a stronger correlation.

4.2 Trends in Developers

The second of the exploratory plots analyzes the maximum concurrent user count a studio has attained from all of their games. The aim was to see if the developers had increased chances of their game going viral if they released more games. To plot this, a technique similar to genres was used to split developers.

```
games_developers <- games_clean_cols |>
  separate_longer_delim(cols=Developers, delim=",") |>
  # Many developers have their name in the form: (developer_name), Inc.
  # Thus, separate_longer_delim collects ' Inc.' as a developer so we need to remove it.
  filter(Developers != " Inc.")

max_ccu_developer <- games_developers |>
```

```
  group_by(Developers) |>
  summarize(games_count=n(),max_CCU=max(`Peak CCU`)) |>
  arrange(desc(max_CCU)) |>
  slice_head(n=10)|>
  ggplot(aes(x=fct_reorder(Developers,games_count), y=max_CCU)) +
  geom_col(fill="#0178ad") +
  geom_text(aes(label=games_count),hjust=-0.2,size=3.5,color="#444") +
  coord_flip() +
  dev_labs

max_ccu_developer
```



It's apparent that although Valve has gotten the highest player counts with a total of 35 games, the actual number of games released by them had no effect on achieving a high CCU. This is because PUBG Corporation has released only 1 game but has higher player counts than developers with 10 games released. As such, the virality of a game depends more on the content of the game and the reputation of the developer, rather than chance.

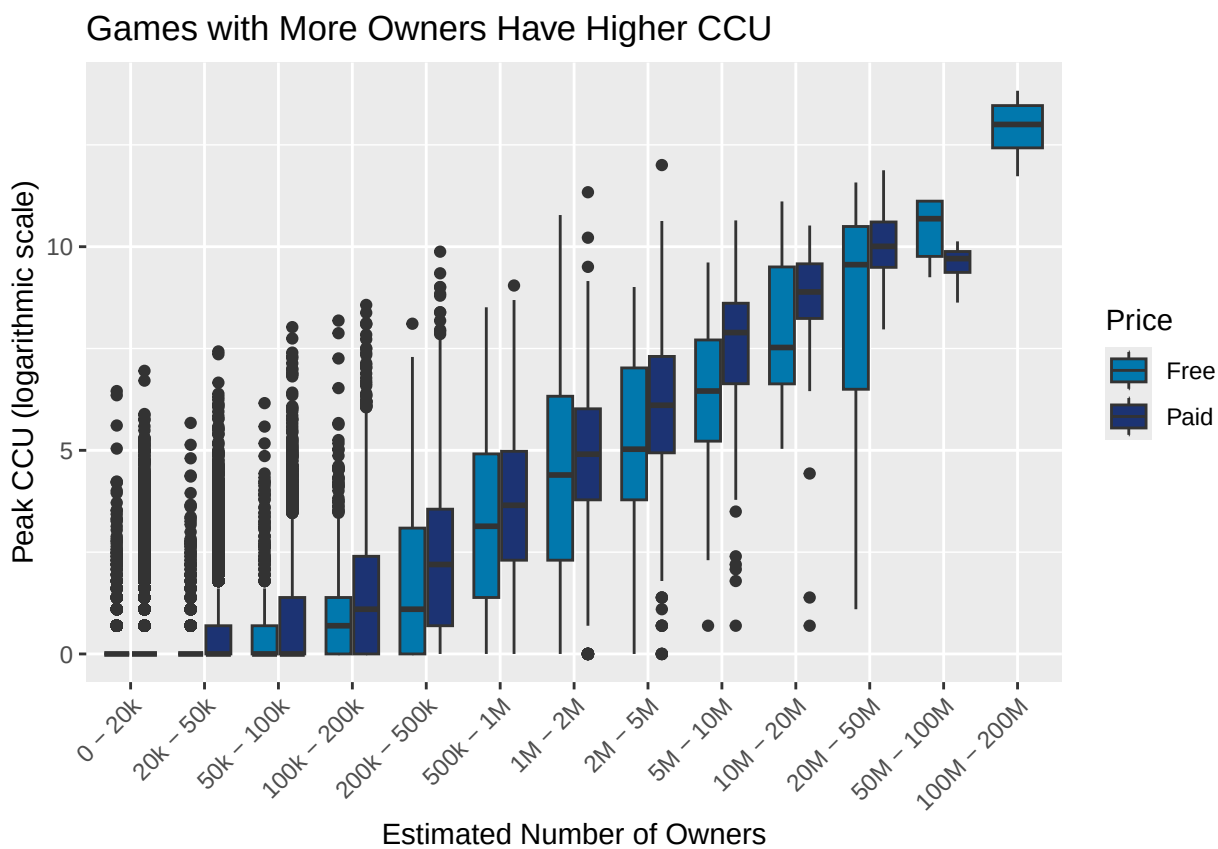
5 Model Plots

5.1 Number of Owners vs Peak CCU

In order to determine trends in `Peak CCU`, the number of `Estimated owners` was plotted against it. The goal was to confirm that a higher owner count means a higher concurrent user count, which intuitively makes

sense.

```
owners_v_ccu <- games_clean_cols |>
  ggplot(aes(
    x=fct_reorder(`Estimated owners`, `Estimated owners high`),
    y=log_ccu,
    fill=Free
    )) +
  geom_boxplot() +
  scale_fill_manual(values=c("#0178ad","#1c3373")) +
  theme(axis.text.x = element_text(vjust = 1, hjust = 1, angle=45)) +
  owners_labs

owners_v_ccu
```

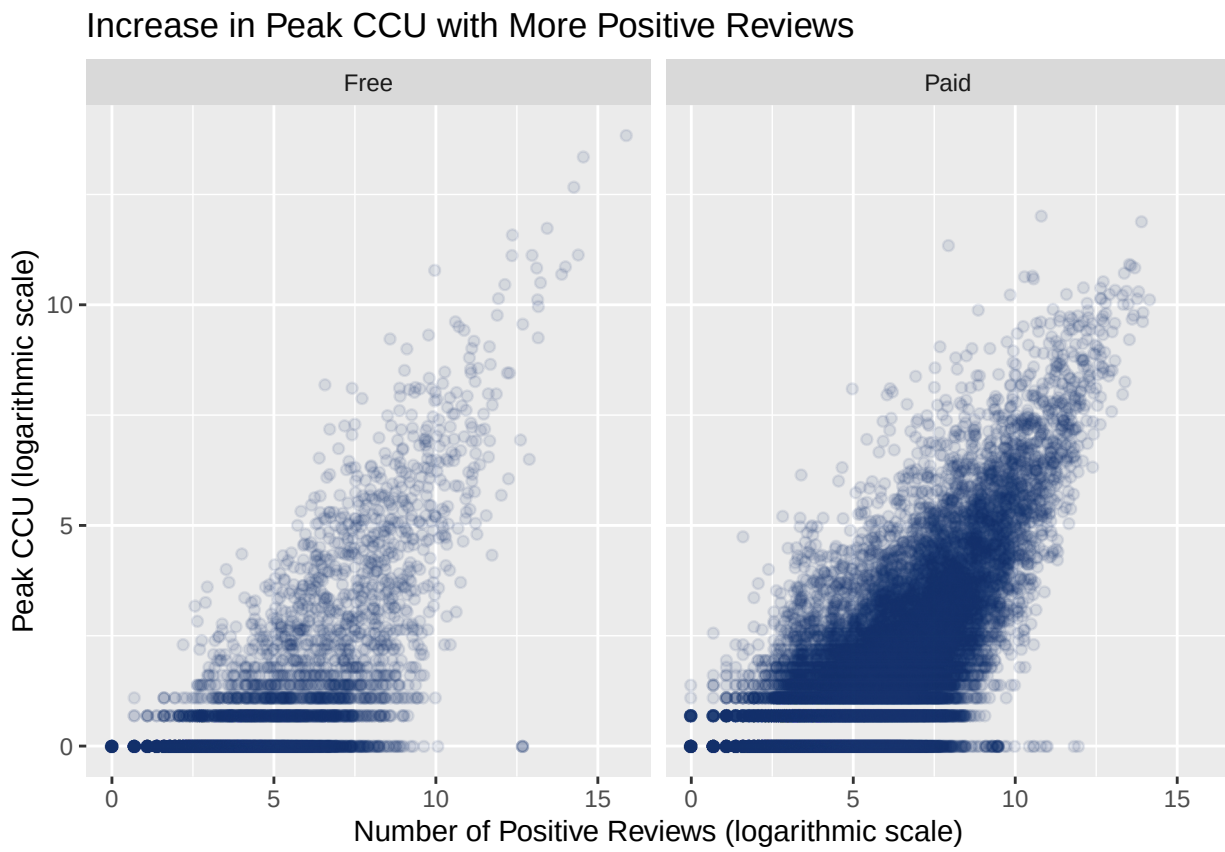


As expected, the owner count and `Peak CCU` are directly proportional, resulting in a smooth line of increasing medians. It's interesting to note the difference in medians among free and paid games. Paid games have higher medians in each category, except for the last two. This suggests that for a game to reach extreme levels of popularity, it must be free. Again, that is thanks to the allure of there being no monetary cost to play.

5.2 Positive Reviews vs Peak CCU

User sentiment plays a large role in the success of a game, so the number of positive reviews on a game could be an indicator to its popularity.

```r
pos_v_ccu <- games_clean_cols |>
  ggplot(aes(x=log(Positive), y=log_ccu)) +
  # Geom_point_rast was used to rasterize the dots for easier viewing in PDF form
  geom_point_rast(color="#1c3373", alpha=0.1) +
  facet_wrap(~Free) +
  positive_labs

pos_v_ccu
```



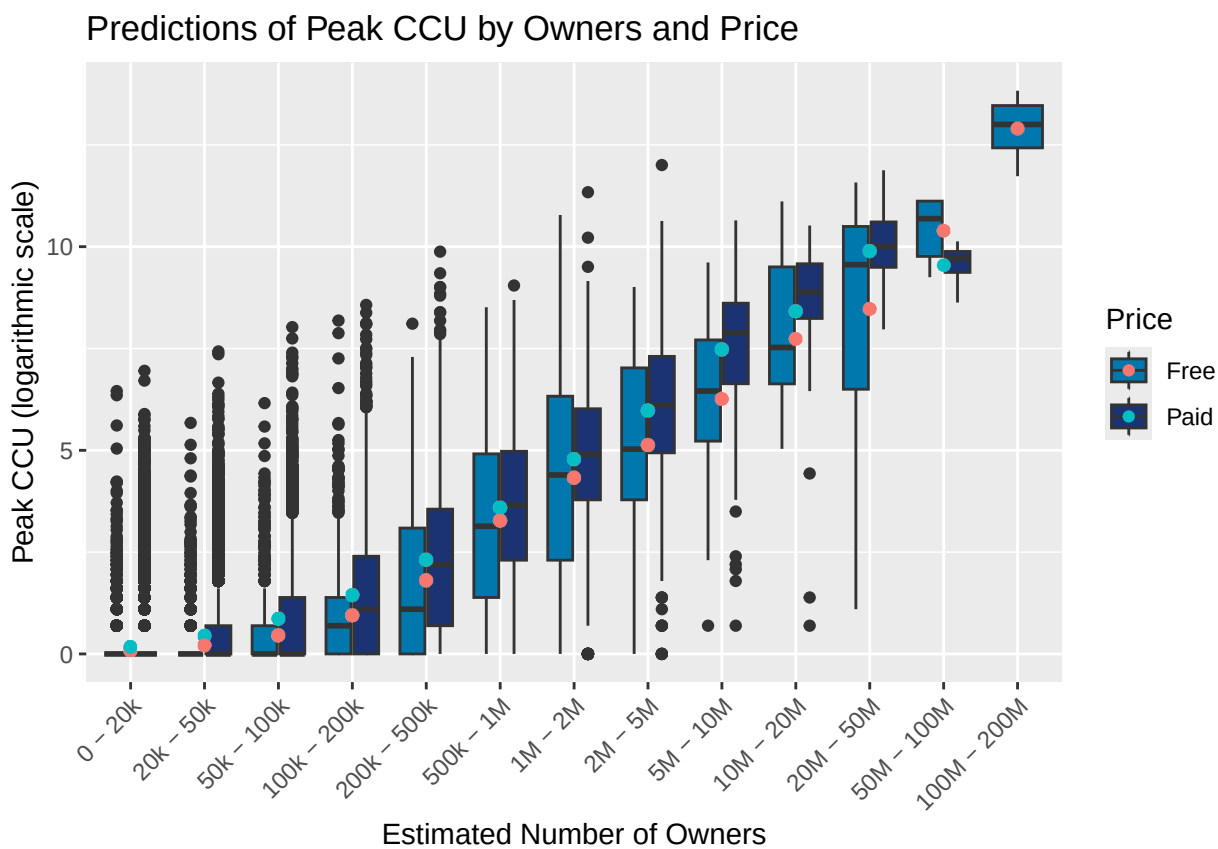
The plot shows a strong linear relationship, justifying that more positive reviews suggest a higher peak player count. However, there are many games with a high number of positive reviews but very low `Peak CCU`. This is because of some missing values but also because there are many games with low `Peak CCU` meaning that success is rare. It's also worth noting that the free games have a more tight spread than paid games, so their `Peak CCU` is more predictable.

6 Exploratory Linear Models

6.1 Linear Model 1

The first of the linear models is based on the number of owners. As seen before, there is a strong correlation between the number of owners and `Peak CCU`, so the number of owners is a strong predictor. Additionally, the price category was found to have a relationship with `Peak CCU` so that will be used as well.

```r
lm_model <- linear_reg() |> set_engine("lm")
wf_owners <- workflow()|>
  add_model(lm_model)|>
  add_formula(log_ccu ~ `Estimated owners`*Free)
model1 <- fit(wf_owners, data=games_clean_cols)
m1_aug <- augment(model1, new_data=games_clean_cols)

lin_model_1 <- games_clean_cols |>
  ggplot(aes(
    x=fct_reorder(`Estimated owners`, `Estimated owners high`),
    y=log_ccu,
    fill=Free)) +
  geom_boxplot() +
  scale_fill_manual(values=c("#0178ad","#1c3373")) +
  theme(axis.text.x = element_text(vjust = 1, hjust = 1, angle=45)) +
  geom_point_rast(data=m1_aug, aes(x=`Estimated owners`, y=.pred,color=Free)) +
  lm1_labs

lin_model_1
```

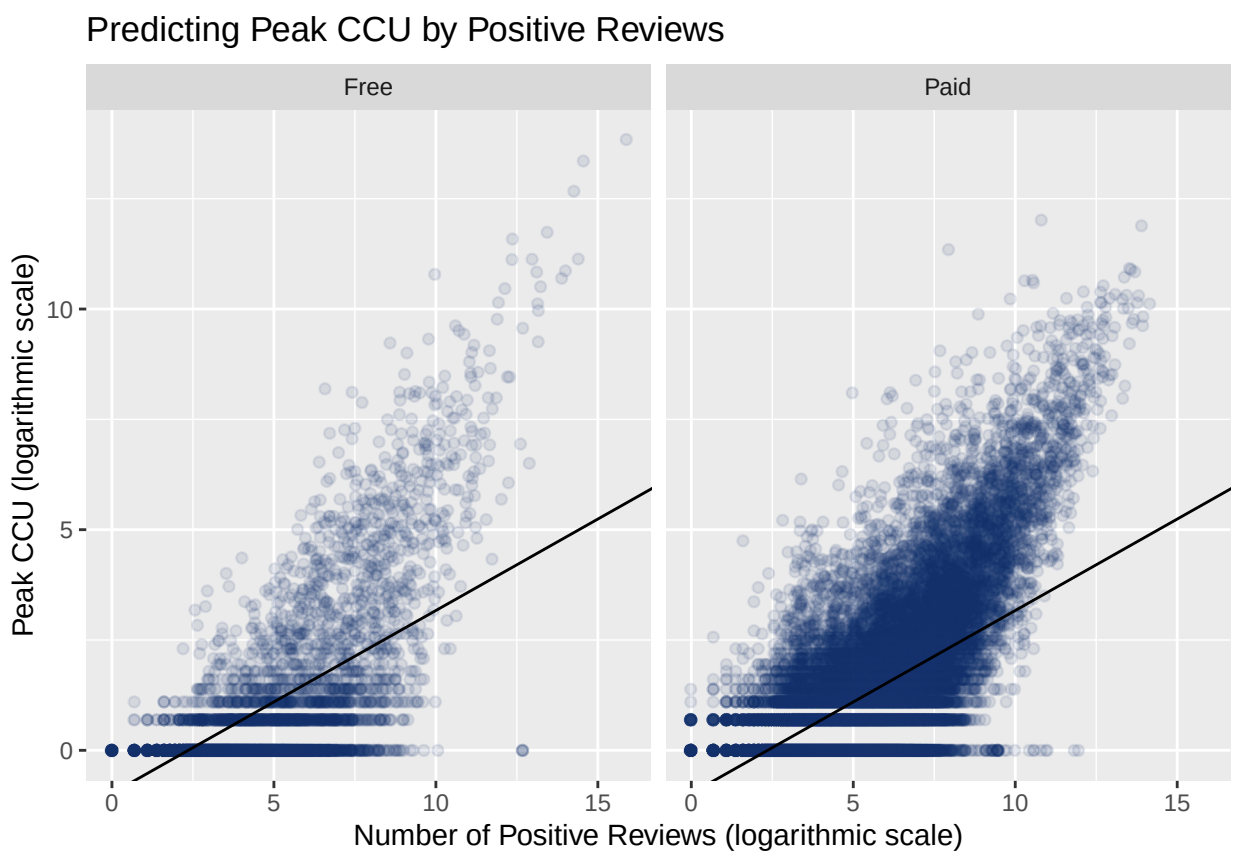


The predictions are quite accurate and they show a trend identical to that of the graph. Using the interaction of `Estimated owners` and `Free` made it so that the model could distinguish between free and paid games within each category, to provide accurate predictions for each. As a result, it can be concluded that the estimated owners and whether a game is free or not are good predictors of the `Peak CCU` of a game. For this model, the R^2 is 0.536, which isn't the greatest but it's important to remember the large variation in steam games data which makes it very hard to predict.

6.2 Linear Model 2

The second linear model relies on the number of positive reviews to predict `Peak CCU`. As shown before, it's clear that `Positive` has a linear relationship with `Peak CCU` because naturally, a game with more positive reviews will have a better reputation and thus will attract more players.

```
wf_positive <- workflow()|>
  add_model(lm_model)|>
  add_formula(log_ccu~ log(Positive))

model2 <-  fit(wf_positive, data=games_clean_cols)
m2_aug <- augment(model2, new_data=games_clean_cols)

b <- as.numeric(coef(extract_fit_engine(model2))[1])
m <- as.numeric(coef(extract_fit_engine(model2))[2])

lin_model_2<- games_clean_cols |>
  ggplot(aes(x=log(Positive), y=log_ccu)) +
  geom_point_rast(color="#1c3373", alpha=0.1) +
  facet_wrap(~Free) +
  geom_abline(intercept=b, slope=m)+
  lm2_labs

lin_model_2
```



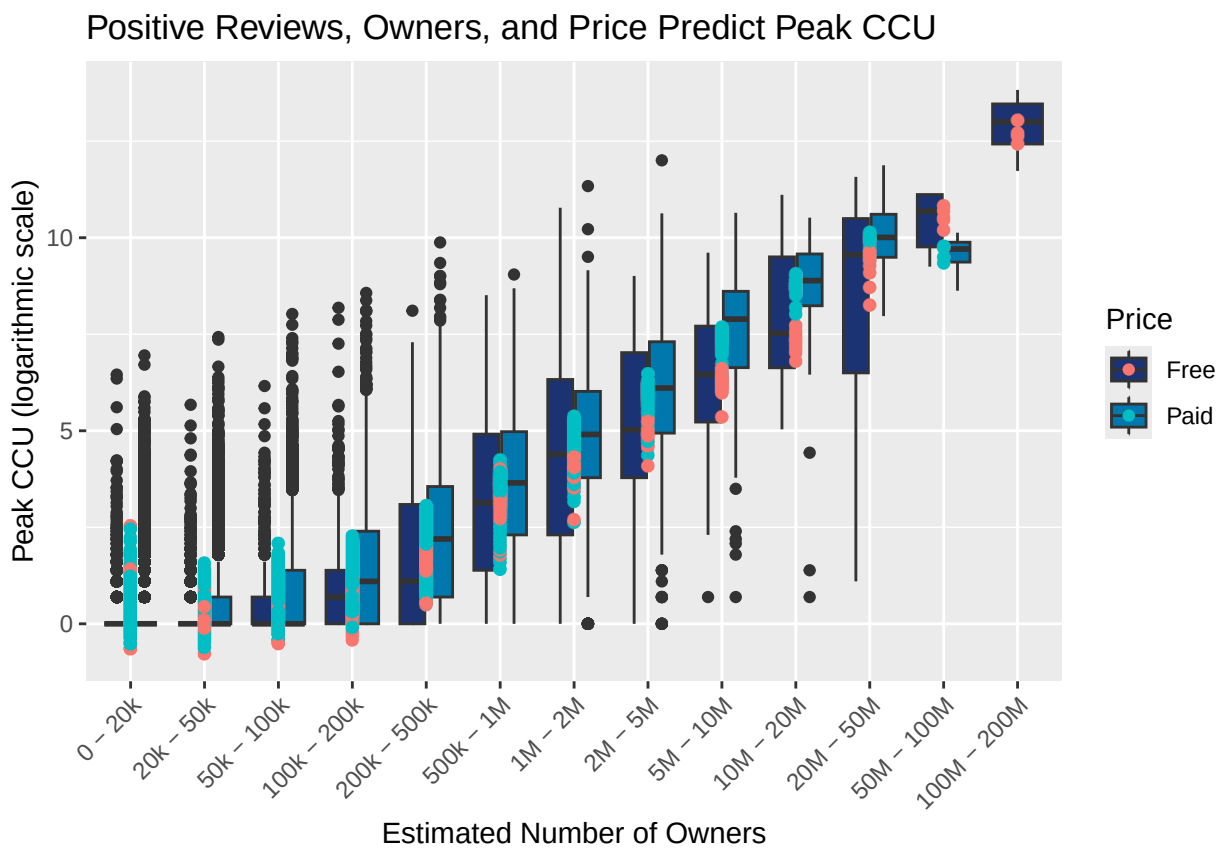
At first glance, this prediction seems very far off, however, the moderate slope is necessary to accurately predict unpopular games without getting a negative value. However, it is not as good at predicting the `Peak`

CCU of highly reviewed games. The R^2 of this model is 0.502. Again, the randomness of games is the cause for a low R^2.

7 Final Model

Finally, after many iterations, the final model I created uses `Positive`, `Estimated owners`, `Free`, and the interaction between `Estimated owners` and `Free` to predict `Peak CCU`. The model is very similar to the first linear model, however, with the addition of `Positive` as a predictor, it is more accurate.

```
wf_final <- workflow()|>
  add_model(lm_model)|>
  add_formula(log_ccu ~ log(Positive)+`Estimated owners`*Free)

set.seed(100)
games_split <- initial_validation_split(games_clean_cols, prop=c(0.6,0.2))
games_training <- games_split |> training()
games_validation <- games_split |> validation()
games_testing <- games_split |> testing()

training_fit <- wf_final |> fit(games_training)
validation_prediction <- training_fit |> predict(games_validation)

games_final <- last_fit(wf_final, split=games_split)
model_final<- games_final |> extract_workflow()
m_aug <- augment(model_final, new_data=games_clean_cols)

final_model <- games_clean_cols |>
  ggplot(aes(
    x=fct_reorder(`Estimated owners`, `Estimated owners high`),
    y=log_ccu,
    fill=Free)) +
  geom_boxplot() +
  scale_fill_manual(values=c("#1c3373","#0178ad")) +
  theme(axis.text.x = element_text(vjust = 1, hjust = 1, angle=45)) +
  geom_point_rast(data=m_aug, aes(x=`Estimated owners`, y=.pred,color=Free))+
  final_labs

final_model
```



This model combines the two previous ones to provide a range of predictions for each category of owners. The R^2 is 0.62 which is much better than previous models. This model was chosen over others because it takes three things into consideration: the amount of owners, the price, and the sentiments of users on the game. These three things play a large role in how viral a game is since a viral game is easily accessible and is likeable by many people.

8 Conclusion

This analysis found that games that are published on Steam are more likely to go viral if the game is likable and free, and thus gets many owners. That is, games with many positive reviews get seen more by people and if it is free, more people are enticed to play it. That being said, in the lower categories of owners, paid games seem to see larger player bases on average which is likely due to a sense of 'loyalty' that the user feels for making a monetary commitment. Lastly, many of the graphs are concentrated at the bottom meaning that the vast majority of games do not see success which could be for a variety of reasons.

9 Limitations

Due to privacy reasons, Steam does not provide owner counts for their games. Because of that, services like steamspy and steamdb make estimates based on public profiles, but these estimates can have a lot of error. This caused some inconsistencies in the data leading to possible errors in trends. Additionally, the virality of a game depends on some factors that could not be present in the data. This includes how relevant the game is to current trends and pop culture, how well it is advertised, and how unique it is in its gameplay. Arguably, these factors combined make a larger impact on the success of a game than those analyzed here.